AI Compiler Performance Engineering

Auto-Loops Books — ch26

Contents

1 Production Deployment and Engineering Best Practices **1**

1.1 env packaging 3

1.2 multi HW ops 5

1.3 pitfalls 6

1.4 scheduling 7

1.5 Key Takeaways 9

1.6 Conclusion 10

Bibliography **11**

Chapter 1

Production Deployment and Engineering Best Practices

Shipping a decode compiler is not shipping a Git tag—it is shipping *reproducible artifacts* (fingerprints, containers, driver matrices) that replay the same bytes/token and launches/token counters your tuning loop proved in CI (Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; Chen, 2020; Chen and YiRage Contributors, 2026). Chapters 22–25 built profiling, gates, LLM/MoE specialization, and automated search; this chapter operationalizes **production deployment** across GPU, CPU, and NPU fleets without letting environment drift erase compiler wins (AMD, 2024b,a; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024).

TaxBreak dense Llama-3.2-1B averages 847.5 kernel launches per output token on H100; OLMoE-1B/7B averages 9,305 (Vellaisamy *et al.*, 2026). Memory-Floor CUDA Graphs on Qwen-2.5-7B (BS=1, ctx=2048) yields 1.259× on H100 by removing host overhead—yet production still fails when packaging omits graph warmup or pins the wrong driver (Chen, 2026; NVIDIA, 2024). Production engineering translates compile-time legality into *fleet-time* legality (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Four practices anchor the chapter: **environment packaging**, **multi-hardware operations**, **production pitfalls**, and **heterogeneous scheduling** (Davies *et al.*, 2025; Chen, 2020; Chen and YiRage Contributors, 2026; AMD, 2024b,a; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen, 2026; Vellaisamy *et al.*, 2026; SemiAnalysis, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Part VIII (Chapters 28–30) assumes these baselines when separating framework, runtime,

and compiler responsibilities (Chen and YiRage Contributors, 2026; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2020; Liu *et al.*, 2024; Chen, 2026; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Production maturity means the same decode cell run in CI is one command away in staging and prod (Chen, 2026; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). When that link breaks, "compiler regressions" are often packaging or scheduling mistakes (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Ownership RACI: compiler engineers sign fingerprint manifests; release engineering signs container promotion; SRE signs fleet config; framework owners sign batching invariants (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; Chen, 2026; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Blurred ownership produces dashboards nobody trusts (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Anti-patterns for production: promoting schedules without staging replay; mixing prefill and decode fingerprints; autoscaling on CPU utilization; silent eager fallback after driver upgrade (NVIDIA, 2024; Chen, 2026; Kwon *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Each section names the fix (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Compliance and audit trails: retain profile artifacts and packaging manifests long enough to explain quarter-over-quarter decode SLO shifts (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

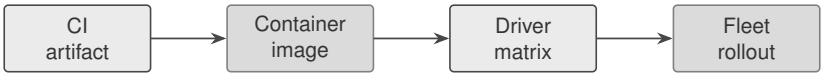


Figure 1.1: Environment packaging: CI compiler artifacts → immutable container → pinned driver matrix → staged fleet rollout.

Package layer	**Must pin**
Compiler stack	git SHA, pass flags, backend registry
Runtime deps	CUDA/cuDNN, ROCm, oneDNN, XDNA toolkit
Model assets	weights hash, tokenizer, paging policy
Profile harness	decode cell script + bucket list

Table 1.1: Production packaging layers for multi-hardware decode fleets.

Regulators and customers increasingly ask *why* latency changed—fingerprints answer that question (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

1.1 env packaging

Environment packaging binds compiler git SHA, LLVM/Triton/CUDA toolkit versions, and schedule fingerprints into deployable units (Chen, 2020; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). The anti-pattern is "works in CI" containers that omit warmup scripts, graph capture caches, or model-weight layout flags present in the tuning cell (Chen, 2026; NVIDIA, 2024; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Multi-hardware fleets package *per-tier* images: Hopper decode, MI300 decode, XDNA edge, CPU fallback—shared YAML env vars are convenient and dangerous (AMD, 2024b,a; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen, 2026; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2024,

2022). Rollout attaches SBOM + fingerprint manifest so on-call can diff a regressed pod in minutes (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Deploy and packaging pipelines should fail closed when fingerprint files are missing, checksums mismatch, or decode harness scripts differ from CI (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). "Best effort" deploy is how illegal graph capture reaches production (NVIDIA, 2024; Chen, 2026; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Secrets and model weights belong in the same manifest chain as compiler artifacts—rotating weights without updating fingerprints is a common silent regression (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; Chen, 2026; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Package deploy hooks that re-run a smoke decode cell before accepting traffic (Chen, 2026; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Multi-hardware delta. GPU images ship CUDA Graph warmup jobs; NPU images ship static compile bundles; CPU images ship ORT/TorchInductor session plans—same model card, different immutable artifacts (NVIDIA, 2024; AMD, 2024b,a; Goodfellow *et al.*, 2016; Chen, 2020; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen, 2026; Vellaisamy *et al.*, 2026; SemiAnalysis, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Example 1.1.1. Deploy promotes fingerprint `fp-8f3a` only when CI decode cell at ctx$\in \{512, 2048, 8192\}$ matches golden bytes/token within ϵ (Chen, 2026; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022).

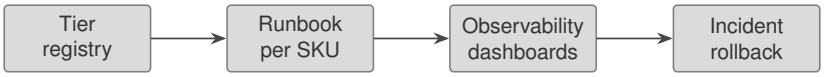


Figure 1.2: Multi-hardware operations: SKU registry → runbooks → decode counter dashboards → fingerprint rollback.

1.2 multi HW ops

Multi-hardware operations keep production honest after deploy (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b,a; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen, 2026; Vellaisamy *et al.*, 2026; SemiAnalysis, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Dashboards must show bytes/token, launches/token, and ms/token *per tier*—aggregating GPU and NPU into one chart hides fallback paths (Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Runbooks document: legal context buckets, graph capture warmup, paging limits, MoE routing flags, and rollback fingerprints (Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). On-call steps start with "diff packaging manifest" not "retune tiles" (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

SRE collaboration: compiler teams own fingerprint promotion; SRE owns fleet capacity and incident response; framework teams own batching—escalation paths must be explicit (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; Chen, 2026; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Review gate. Production promotion requires tier-specific decode-cell replay in staging that mirrors driver and power caps (Chen, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

1.3 pitfalls

Production pitfalls recur across fleets (Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022):

1. **Driver/toolkit skew** — CI green, prod slow; graph capture silently falls back to eager (NVIDIA, 2024; Chen, 2026; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

2. **Wrong context bucket** — schedule legal at ctx=2048 spills at ctx=8192 (Chen, 2026; Dao *et al.*, 2022; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; AMD, 2024b).

3. **Benchmark theater** — prefill FLOPs celebrated while decode bytes/token flatlines (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; AMD, 2024b).

4. **Cross-tier promotion** — GPU fingerprint replayed on NPU without reprofile (AMD, 2024b,a; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen, 2026; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Postmortems should log counter deltas, packaging diffs, and routing histograms—not only "kernel X slower" (Davies *et al.*, 2025; Liu *et al.*, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Chen, 2026; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Lessons feed back into Chapter 22 gates and Chapter 25 search budgets (Chen, 2020; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Pitfall drills: game-day exercises that upgrade drivers, shrink memory, or skew routing—teams rehearse fingerprint rollback before incidents (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; AMD, 2024b). Lessons learned belong in runbooks, not slide decks (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Chen, 2026; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Operation teams should track *packag* drift— undeclared env vars, sidecar injections, and mutating init scripts—as first-class incident causes (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Production operation metrics include deploy rollback rate, not only availability (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Chen, 2026; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

1.4 scheduling

Heterogeneous scheduling places prefill pools, decode pools, MoE expert shards, and CPU/NPU fallbacks on distinct fleet slices (Davies *et al.*, 2025; Liu *et al.*, 2024; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b,a; Goodfellow *et al.*, 2016; Chen, 2026; Vellaisamy *et al.*, 2026; SemiAnalysis, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Compiler packaging must tag which fingerprints are valid on which pool—schedulers should not load Hopper graphs onto bandwidth-starved SKUs (Chen, 2026; SemiAnalysis, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Kubernetes-style orchestration needs decode-aware autoscaling: scale on ms/token and queue depth, not CPU percent (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; Chen, 2026; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Prefill bursts and decode steady-state belong on different node pools with different compiler artifacts (Davies

et al., 2025; Liu *et al.*, 2024; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020, 2026; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

ClusterTopology-style metadata (Chapter 24) informs which compilers may fuse across NUMA/NVLink boundaries; scheduling respects those edges (Chen, 2020; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen, 2026; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; AMD, 2024b). Canary pools per schedule fingerprint remain mandatory—fleet schedulers amplify mistakes faster than laptops (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Chen, 2026; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Fleet schedul policies should expose which fingerprint each replica runs—debugging p99 without that label wastes hours (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Chen, 2026; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Schedul changes (pool resize, spot preemption, NUMA rebinding) trigger mandatory decode-cell smoke tests when compiler residency is NUMA-sensitive (Goodfellow *et al.*, 2016; Chen, 2020; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Kwon *et al.*, 2023; Chen, 2026; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; AMD, 2024a; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; AMD, 2024b).

Cost-aware fleet scheduling trades spot GPU pools against SLO headroom—compiler fingerprints validated on stable pools first, spot second (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Chen, 2026; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Goodput per dollar still defers to bytes/token per dollar under decode load (Davies *et al.*, 2025; Goodfellow *et al.*, 2016; Chen, 2026; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

1.5 Key Takeaways

- **Package the whole compile, not just the model.** A deployable unit must bind the compiler git SHA, the LLVM/Triton/CUDA toolkit versions, and the schedule fingerprints into one immutable image, so a rebuild reproduces the exact kernels (Chen, 2020; Chen and YiRage Contributors, 2026). Without that pinning, a driver or toolkit drift silently changes the emitted schedule and the decode numbers move for reasons no one can trace (Davies *et al.*, 2025; Chen, 2026).

- **Operate multi-hardware with runbooks, dashboards, and rollback.** Production stays honest after deploy only with per-tier runbooks, counter dashboards tracking bytes/token and launches/token, and fingerprint-keyed rollback to a known-good schedule (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026). The same model on GPU, CPU, and NPU has different failure modes, so the operational tooling must be tier-aware rather than one-size-fits-all (AMD, 2024b,a).

- **Know the recurring pitfalls.** Driver skew, shape-bucket mismatch, benchmark theater (microbench wins that never move ms/token), and cross-tier promotion of a schedule tuned for the wrong SKU recur across fleets (NVIDIA, 2024; Vellaisamy *et al.*, 2026). Each is invisible to a FLOPs dashboard and visible only to a decode-cell regression on the actual target (Chen, 2026; Davies *et al.*, 2025).

- **Schedule prefill and decode as distinct fleet slices.** Heterogeneous scheduling places prefill pools, decode pools, MoE expert shards, and CPU/NPU fallbacks on separate slices, each with its own fingerprints and decode-aware autoscaling (Davies *et al.*, 2025; Kwon *et al.*, 2023). Forcing both phases onto one uniform pool wastes the 10× capacity skew that disaggregation exists to exploit (Liu *et al.*, 2024; Chen, 2026).

- **Hold SLOs to decode goodput, not peak FLOPs.** Production service-level objectives must track bytes/token and launches/token at the real operating point, because those are what predict ms/token and p99 (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). A green FLOPs or utilization dashboard that ignores them is the most common way a regression ships unnoticed (Chen, 2026; Goodfellow *et al.*, 2016).

1.6 Conclusion

Production deployment and engineering best practices turn compiler wins into fleet contracts (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Package reproducibly, operate per tier, document pitfalls, and schedule heterogeneously—then Chapter 27 explores where silicon trends stress those contracts (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen, 2026; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; AMD, 2024b). Part VIII implements framework–runtime–compiler co-design assuming these production gates held (Chen and YiRage Contributors, 2026; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2020; Liu *et al.*, 2024; Chen, 2026; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Gate merges on buffer accounting and reproducible decode cells—never on laptop demos alone (SemiAnalysis, 2024; Chen, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Bibliography

AMD (2024a). Ai engine technology. `https://www.AMD.com/en/products/adaptive-socs-and-fpgas/technologies/AI-engine.html`. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10

AMD (2024b). AMD XDNA architecture. `https://www.AMD.com/en/technologies/XDNA.html`. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10

Chen, J. (2026). Memory-bound but not bandwidth-limited: The physical AI inference gap in batch-1 LLM decode. *arXiv preprint arXiv:2605.30571*. 44-cell cross-GPU study; CUDA Graphs A/B on H100 and L4. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10

Chen, X. (2020). Compiler-based hardware profiling for software-defined architecture. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10

Chen, X. and YiRage Contributors (2026). Yirage: Multi-backend LLM inference optimization. `https://github.com/chenxingqiang/YiRage`. Yield Revolutionary AGile Engine; extends Mirage with CUDA, CPU, MPS, Triton, NKI, and MLIR backends. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10

Dao, T. *et al.* (2023). Flash-decoding for long-context inference. PyTorch blog and Stanford CRFM technical report. Parallelizes attention along KV sequence; up to 8× decode speedup at long context. 1, 2, 3, 4, 5, 6, 7, 8, 10

Dao, T., Fu, D., Ermon, S., Rudra, A., and Ré, C. (2022). FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*. 1, 2, 3, 4, 5, 6, 7, 8, 10

Davies, M. *et al.* (2025). Liminal: Exploring the frontiers of LLM decode performance. *arXiv preprint arXiv:2507.14397*. Limit study; DeepSeek-V3 provisions 10× more decode than prefill nodes. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10

Goodfellow, I., Bengio, Y., and Courville, A. (2016). *Deep Learning*. MIT Press. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. (2023). Efficient memory management for large language model serving with PagedAttention. *Proceedings of the 29th Symposium on Operating Systems Principles*. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10

Liu, A. *et al.* (2024). Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10

NVIDIA (2022). Nvidia hopper architecture in-depth. NVIDIA Technical Blog. 1, 2, 3, 4, 5, 6, 7, 8, 10

NVIDIA (2024). Cuda graphs. CUDA Programming Guide. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10

SemiAnalysis (2024). Nvidia tensor core evolution: From volta to blackwell. Newsletter. 1, 2, 3, 4, 5, 6, 7, 8, 10

Vellaisamy, P., Tripathi, S., Natarajan, V., Thenarasu, S. S., Blanton, S., and Shen, J. P. (2026). Taxbreak: Unmasking the hidden costs of LLM inference through overhead decomposition. *arXiv preprint arXiv:2603.12465*. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10